SNN 实验阶段性汇报（基于最新代码与最新结果）

日期：2026-06-02

1. 本阶段完成内容

当前项目已形成一个可复现实验闭环，包含：

- `train.py`：训练 + 验证 + checkpoint + 日志与曲线
- `test.py`：加载最佳 checkpoint，在官方测试集上独立评估
- `snn_minimal/data.py`：数据增强、训练集切分 train/val、官方测试集加载
- `snn_minimal/plotting.py`：训练日志与 `train_loss.svg` / `val_acc.svg` 输出

2. 项目整体思路

本项目参考了 SpikingJelly 仓库中 `activation_based` 对神经元LIFNode/IFNode的定义

总体思想是：

1. 使用卷积网络提取空间特征。
2. 在卷积层和全连接层后加入脉冲神经元。
3. 将静态图像沿时间维重复 `T` 次。
4. 让神经元膜电位在时间维上演化。
5. 使用输出层在 `T` 个时间步上的平均发放率作为最终分类结果。

这是一种典型的 rate-based SNN 分类实现。

3. 当前模型原理

3.1 输入方式

把一张静态图像复制到所有时间步：

```
x[1] = x[2] = ... = x[T] = input_image
```

因此，时间动态并不是来自输入本身变化，而是来自脉冲神经元内部膜电位在时间上的积累、泄漏、阈值触发和重置。

3.2 当前实际使用的脉冲神经元

代码支持两类神经元：

- `LIFNode（有泄漏）`
- `IFNode（无泄漏）`

但当前已完成的两组实验均使用：

- `LIFNode`

对应参数文件均为：

- `neuron = lif`

3.3 LIF 神经元基本公式

设：

- `x[t]`：第 `t` 个时间步输入
- `h[t]`：发放前膜电位
- `s[t]`：脉冲输出
- `v[t]`：发放后的膜电位
- `V_th`：阈值
- `V_reset`：重置电位
- `tau`：时间常数

LIF 概念性动力学可写为：

```
h[t] = v[t-1] + (x[t] - (v[t-1] - V_reset)) / tau
s[t] = Theta(h[t] - V_th)
v[t] = V_reset, if s[t] = 1
v[t] = h[t],     if s[t] = 0
```

本项目默认配置：

- `tau = 2.0`
- `V_th = 1.0`
- `V_reset = 0.0`

可近似理解为：

```
h[t] = v[t-1] + (x[t] - v[t-1]) / 2
```

含义：

- 神经元不断积分输入
- 同时膜电位向重置电位泄漏
- 膜电位超过阈值时发放 spike
- 发放后膜电位被重置

3.4 Spike 的定义

脉冲输出是二值变量：

```
s[t] = Theta(h[t] - V_th)
```

其中：

```
Theta(z) = 1, if z >= 0
Theta(z) = 0, if z < 0
```

即只有膜电位达到阈值时，神经元才在该时间步发放 spike。

3.5 反向传播中的 surrogate gradient

由于硬阈值函数 `Theta(.)` 几乎处处不可导，不能直接用标准反向传播训练，因此本项目使用了 SpikingJelly 的替代梯度函数：

- `surrogate.ATan()`

其作用是：

- 前向传播仍产生二值 spike
- 反向传播用平滑近似函数代替硬阈值导数

这样才能让梯度跨过 spike 函数，实现端到端训练。

3.6 输出读出方式

输出层同样是脉冲神经元层，模型最终输出不是 ANN 常见 logits，而是输出层在多个时间步上的平均发放率：

```
y = (1 / T) * sum_t s_out[t]
```

最后得到形状 `[N, 10]` 的分类输出，再做：

```
pred = argmax(y, dim=1)
```

4. 当前网络结构

模型定义文件：`snn_minimal/models/conv_snn.py`

4.1 通用结构

模型由两部分组成：

- 编码/特征提取部分 `encoder`
- 分类头 `classifier`

卷积块基本模式：

```
Conv2d -> BatchNorm2d -> SpikingNeuron -> Pooling
```

分类头基本模式：

```
Flatten -> Dropout -> Linear -> SpikingNeuron -> Dropout -> Linear -> SpikingNeuron
```

4.2 MNIST 网络结构

默认参数：

- `timesteps = 4`
- `channels = 64`
- `hidden_dim = 256`

结构：

1. 输入：`[N, 1, 28, 28]`
2. 时间展开：`[T, N, 1, 28, 28]`
3. `Conv2d(1 -> 64, 3x3)` + `BN` + `LIF`
4. `MaxPool2d(2)`
5. `Conv2d(64 -> 128, 3x3)` + `BN` + `LIF`
6. `MaxPool2d(2)`
7. `Flatten`
8. `Dropout(0.25)`
9. `Linear(128 * 7 * 7 -> 256)`
10. `LIF`
11. `Dropout(0.25)`
12. `Linear(256 -> 10)`
13. `LIF`
14. 对时间维求平均，输出 `[N, 10]`

4.3 CIFAR-10 网络结构

默认参数：

- `timesteps = 4`
- `channels = 64`
- `hidden_dim = 256`

结构：

1. 输入：`[N, 3, 32, 32]`
2. 时间展开：`[T, N, 3, 32, 32]`
3. `Conv2d(3 -> 64, 3x3)` + `BN` + `LIF`
4. `Conv2d(64 -> 64, 3x3)` + `BN` + `LIF`
5. `MaxPool2d(2)`

6. `Conv2d(64 -> 128, 3x3)` + `BN` + `LIF`
7. `Conv2d(128 -> 128, 3x3)` + `BN` + `LIF`
8. `MaxPool2d(2)`
9. `Flatten`
10. `Dropout(0.25)`
11. `Linear(128 * 8 * 8 -> 256)`
12. `LIF`
13. `Dropout(0.25)`
14. `Linear(256 -> 10)`
15. `LIF`
16. 对时间维求平均，输出 `[N, 10]`

5. 训练与测试流程

5.1 训练脚本（`train.py`）

每个 epoch 流程：

1. 从训练集读取 batch
2. 前向传播得到输出发放率
3. 将标签转为 one-hot
4. 计算损失
5. 反向传播并更新参数
6. 调用 `functional.reset_net(model)` 重置神经元状态
7. 在验证集上评估
8. 保存最新 checkpoint 与最佳 checkpoint
9. 记录训练日志与曲线

5.2 损失函数

当前损失函数不是交叉熵，而是：

```
loss = MSE(output_firing_rate, one_hot_label)
```

原因：

- 与 firing-rate 读出方式自然匹配

5.3 为什么每个 batch 后必须 reset

SNN 与 ANN 的关键差别是神经元内部有状态。

如果一个 batch 结束后不清除膜电位，下一个 batch 会继承前一个 batch 的状态，导致训练和测试结果错误。

因此每个 batch 后都调用：

```
functional.reset_net(model)
```

5.4 测试脚本（`test.py`）

该脚本会：

1. 读取 run 文件夹下的 `args.json`
2. 根据训练参数重新构造模型
3. 加载 `checkpoint_best.pt`
4. 在官方测试集上评估
5. 写出 `test_metrics.json`

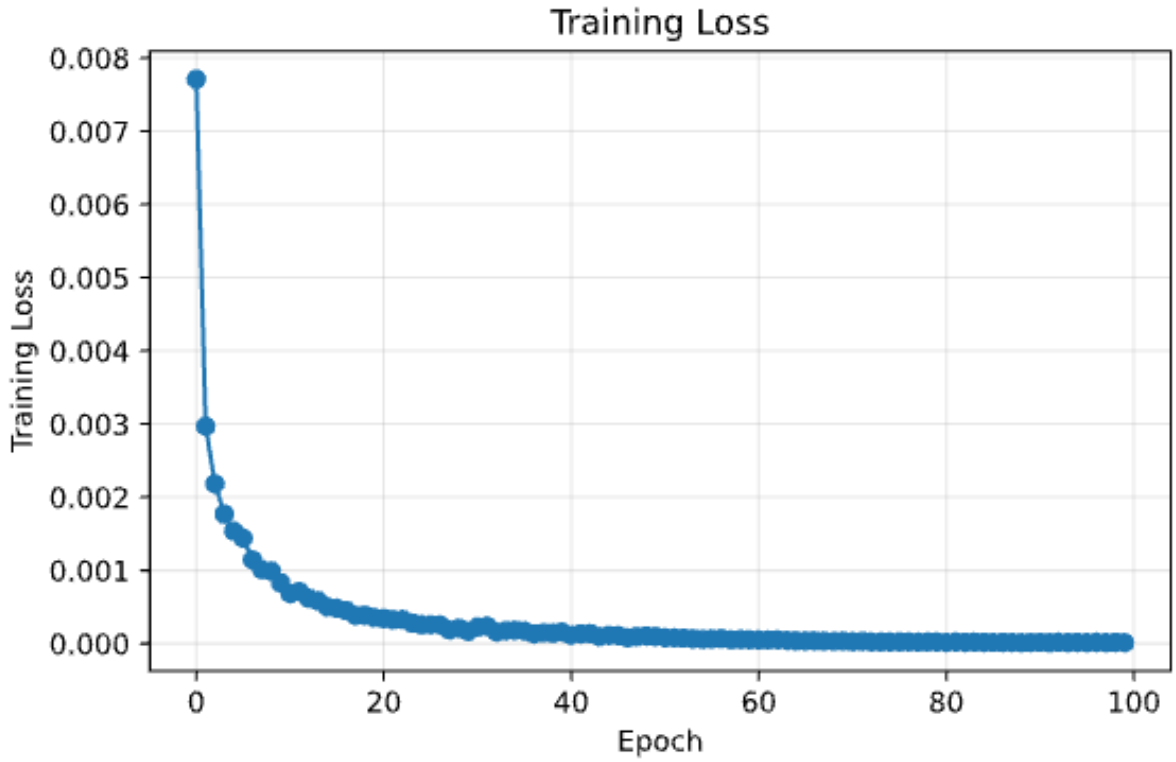
6. 最新实验结果（基于当前 runs）

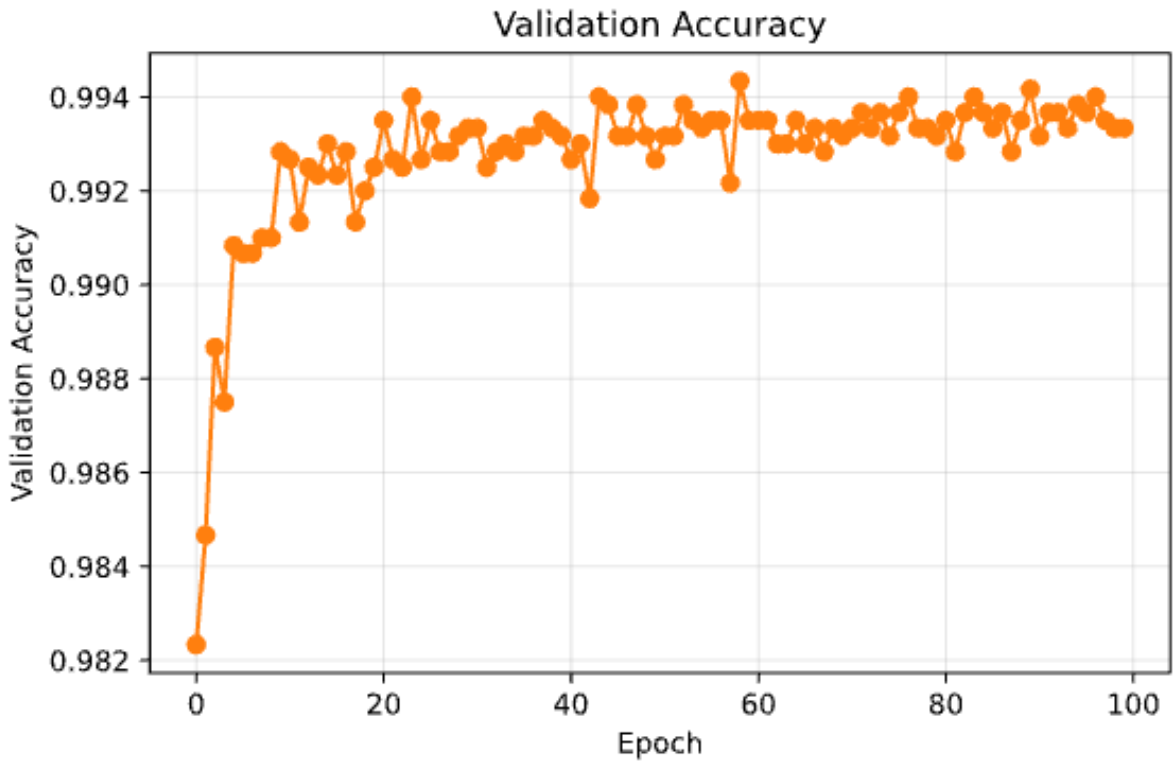
主要训练配置：

- `timesteps=4`
- `channels=64`
- `hidden_dim=256`
- `optimizer=adam`，`lr=1e-3`
- `batch_size=64`
- `amp=true`（CUDA）

6.1 MNIST（最新）

结果目录：`runs/mnist/mnist_20260602_052040/`





来自 `metrics.json`：

- best val acc: **0.9943**（epoch 58）
- final val acc: **0.9933**（epoch 99）
- final train loss: **1.20e-05**

结论：

- MNIST 收敛快且稳定，模型容量对该任务基本充足。

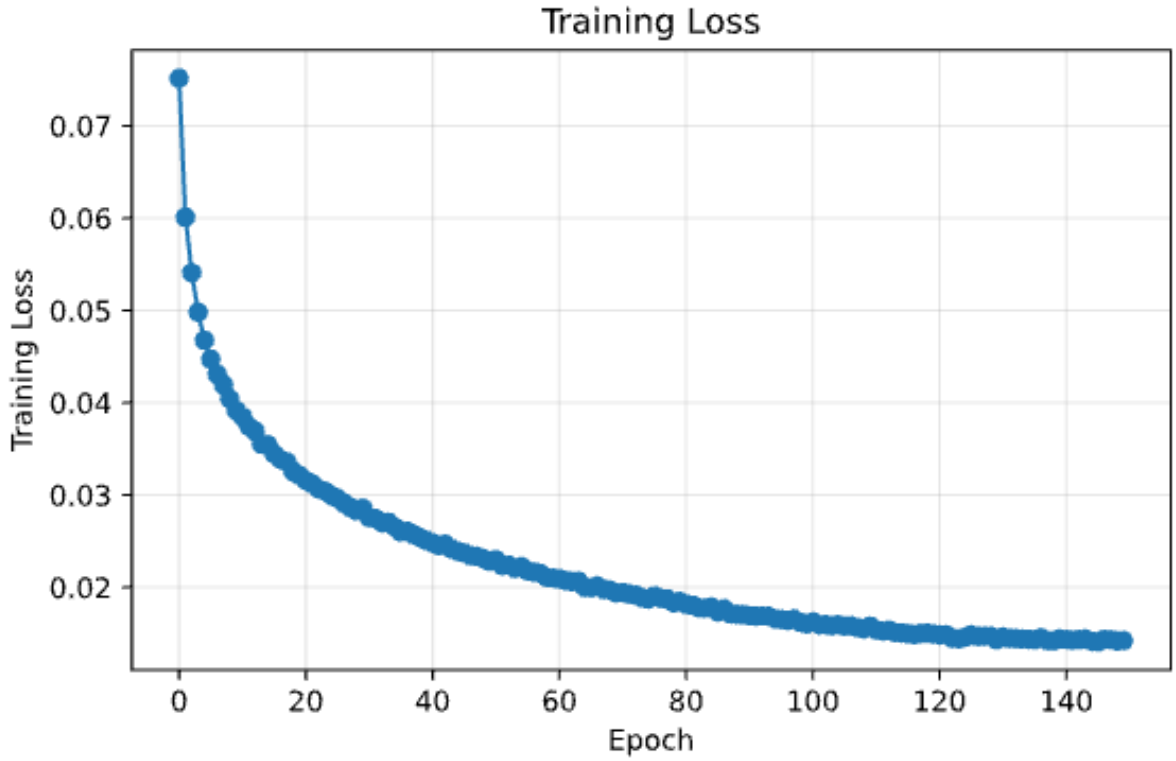
- best 与 final val 差距小（约 0.1 个百分点），后期主要为高位小幅波动。

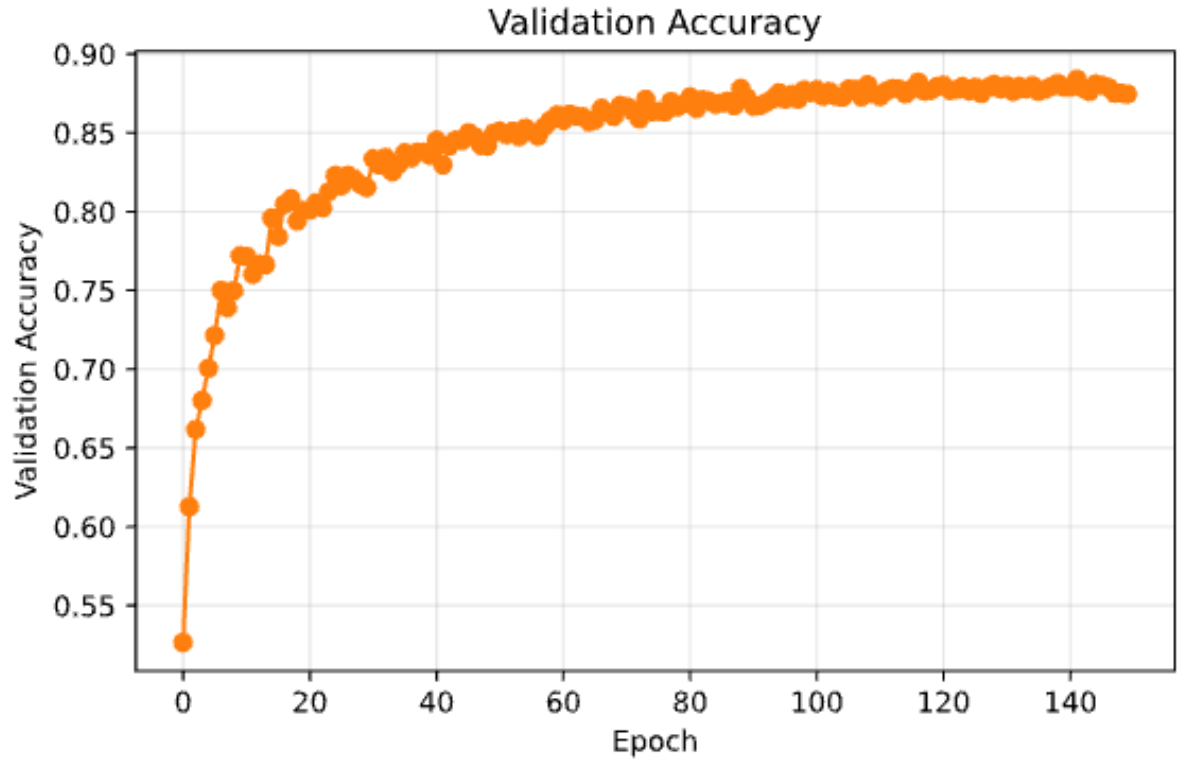
6.2 CIFAR-10（最新）

结果目录：`runs/cifar10/cifar10_20260602_072653/`

`args.json` 显示本次训练配置：

- epochs: **150**
- device: **cuda**
- amp: **true**
- val_ratio: **0.1**
- split_seed: **42**





来自 `metrics.json`：

- best val acc: **0.8840**（epoch 141）
- final val acc: **0.8746**（epoch 149）
- final train loss: **0.01423**

来自 `test_metrics.json`（官方测试集）：

- test acc: **0.8804**
- test loss: **0.01859**
- test samples: **10000**

结论：

- CIFAR-10 取得 **88.04% test acc**，结果稳定且可复现。
- best 与 final 差距约 0.94 个百分点，说明后期存在一定回落，最佳 checkpoint 选择是必要的。

7. 目录与文件说明

7.1 顶层

- `train.py`：训练入口，支持 `--val-ratio` / `--split-seed`
- `test.py`：测试入口，默认评估 `checkpoint_best.pt`
- `README.md`：使用说明与技术细节
- `report.md`：阶段汇报（本文件）
- `requirements.txt`：依赖

7.2 核心模块

- `snn_minimal/data.py`：数据增强与 DataLoader；官方训练集切分 train/val；官方测试集单独加载
- `snn_minimal/engine.py`：`run_epoch`、checkpoint 保存与加载
- `snn_minimal/plotting.py`：日志记录、指标摘要、SVG 曲线
- `snn_minimal/models/conv_snn.py`：模型定义

7.3 运行输出（`runs/...`）

每个 run 目录包含：

- `args.json`
- `checkpoint_latest.pt`
- `checkpoint_best.pt`
- `train.log`
- `metrics.json`
- `train_loss.svg`
- `val_acc.svg`
- `test_metrics.json`（运行 `test.py` 后生成）

8. 当前阶段结论

1. 项目已具备完整可复现实验流程：训练（train/val split）+ 独立测试（official test）。
2. MNIST 结果稳定在高位（val acc 约 99.4%）。
3. CIFAR-10 在当前轻量 Conv-SNN 配置下达到 **88.04% test acc**（latest run）。